

132 - Multi-axis and Non-planar Slicer for Robotic 3D Printing

Client: Louis Lamont / Richard Blackwell / Max Melamed

Allowed groups: 2 groups

Project background & goals

Non-planar slicing for robotic printing generates paths where nozzle orientation and layer height vary continuously across a surface, instead of fixed Z-stacked layers. It requires decomposing a surface into deposition paths, solving tool orientation within reach/joint limits, checking collisions across the full path, and outputting motion the controller can execute. No open-source slicer solves this end-to-end; capable tools are closed-source and cell-specific, or ad hoc Grasshopper definitions that don't generalise. This project builds on that existing logic to develop an open, modular slicer, independent of CAD platform or robot type, with core algorithms built as a standalone library and supporting export for common robot formats.

Project scope

Develop a standalone, open-source non-planar slicing library (not a CAD plugin) that takes a 3D model plus print parameters and outputs multi-axis toolpaths.

In scope: surface segmentation into deposition paths, tool orientation solving, path-level collision checking, an intermediate tool-path representation independent of robot/CAD, and export to ABB RAPID and KUKA KRL with configurable extrusion command logic.

Out of scope: slicing algorithm research beyond adapting known approaches and support for robot platforms beyond RAPID/KRL (architecture should not preclude adding others later).

There is capacity for physical print validation at Composite Sydney's studio if development reaches a satisfactory level.

Project requirements

- Accept common mesh/surface input formats (e.g. STL, OBJ, or NURBS via a CAD interchange format).
- Segment/Slice input geometry into deposition paths using a defined strategy (e.g. isoline/stress-aligned/conformal/tween), with parameters exposed rather than hardcoded.
- Compute tool orientation per path point, constrained by a configurable robot reach/joint-limit model (not tied to one physical arm).
- Detect and flag collisions between tool, part, and robot body across the toolpath.
- Represent toolpaths in an intermediate, robot- and CAD-agnostic data structure.
- Export toolpaths to ABB RAPID and KUKA KRL, with extrusion/flow commands defined as an adjustable, per-controller mapping rather than fixed syntax.
- Provide a way to visualise generated toolpaths (simulation/preview) for verification without requiring physical hardware.
- Structure the codebase as a library with clear module boundaries (segmentation, kinematics/orientation, collision, export), not a monolithic script, so it can be extended or swapped independently.

Required knowledge and skills

Solid programming fundamentals (Python and/or C); comfort with 3D geometry/linear algebra (vectors, transforms, meshes); some exposure to robotics concepts (forward/inverse kinematics, joint limits) is useful but can be learned during the project; interest in computational geometry and/or robotics; ability to work with version control and structure a multi-module codebase; prior exposure to Grasshopper/Rhino or robot programming languages (RAPID/KRL) is a bonus, not required.

We're happy for teams to make responsible use of generative AI tools (Copilot, ChatGPT, Claude, etc.) throughout this project. We'd rather students ship a better slicer faster than avoid the tools on principle. Students should still understand and be able to explain every part of the codebase they submit; AI-assisted code is not a substitute for genuinely learning the geometry, kinematics, and robotics concepts underpinning the project.

Expected outcomes and deliverables

Open-source source code (library any CLI/visualisation tooling) in a public repository; documentation covering architecture, module interfaces, and how to extend segmentation/orientation/export strategies; a user guide for running the slicer on sample geometry and generating RAPID/KRL output; a small set of test cases/sample models demonstrating the pipeline end-to-end in simulation; a short final report or writeup summarising design decisions and known limitations.

Disciplines related to the project

Software Development;Web Application Development;Robotics;Additive Manufacturing

Resources provided

<https://archmanu.com/cpd/talk-series-48/>